

Simple Definition

An AI Agent is a system that can:

- 1. Understand a goal
- 2. Make decisions
- 3. Use tools
- 4. Take actions
- 5. Observe results
- 6. Adjust its behavior to achieve the goal

Unlike a normal LLM that only answers questions, an agent can **act on behalf of a user**.

One-Line Definition

An AI Agent is an LLM-powered system that can **reason, plan, use tools, and take actions autonomously** to achieve a specific goal.

Traditional LLM vs Agent

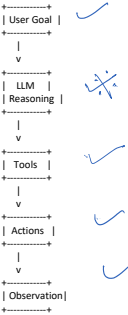
LLM	Agent
Answers questions	Performs tasks ?
Single response	Multi-step workflow
No memory	Can use memory
No tools	Uses tools

Example

LLM
User: What's the weather in Bangalore?
LLM: It is 25°C and cloudy.

Agent
User: Book a trip to Sakleshpur this weekend.
Agent:
1. Searches weather
2. Finds trains/buses
3. Compares hotels
4. Creates itinerary
5. Sends recommendations

Core Components of an Agent



The 5 major components of an AI Agent are:

1. Goal (Objective)

The task the agent needs to accomplish.

Example:
"Find the best flight from Bangalore to Delhi."
Without a goal, the agent doesn't know what to do.

2. Brain (LLM / Reasoning Engine)

This is the decision-making component.

- Examples:
- OpenAI GPT
 - Google Gemini
 - Anthropic Claude
- Responsibilities:
- Understand user intent
 - Think step-by-step
 - Decide which tool to use
 - Generate responses

3. Memory

Stores information across interactions.

Short-Term Memory

Current conversation context.

Long-Term Memory

User preferences, past actions, stored knowledge.

Example:
User prefers vegetarian food.
Agent remembers this when recommending restaurants.

4. Tools

External capabilities the agent can use.

- Examples:
- Web Search
 - Calculator
 - Database
 - Email API
 - Calendar API
 - Weather API

Example:
To answer "What's the weather today?", the agent calls a weather API instead of guessing.

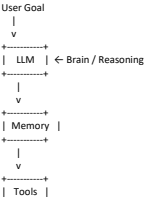
5. Action / Execution Layer

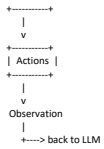
Actually performs tasks.

- Examples:
- Send an email
 - Create a ticket
 - Book a meeting
 - Query a database
 - Update a CRM record

This is what makes an agent different from a chatbot.

How They Work Together





Tool Integration in AI Agents

Tool integration is the ability of an AI agent to connect with and use external systems, APIs, databases, or applications to perform tasks beyond text generation.

Why is Tool Integration Needed?

An LLM alone:

- Cannot access real-time data
- Cannot send emails
- Cannot query databases
- Cannot book meetings
- Cannot call APIs

Tools give the agent these capabilities.

Common Types of Tools

1. Search Tools

Used to get real-time information.

Examples:

- Google Search
- Web Search APIs

User:

Latest AI news

Agent:

→ Calls search tool
→ Summarizes results

2. Database Tools

Used to fetch enterprise data.

Examples:

- SQL Database
- BigQuery
- PostgreSQL

User:

Show yesterday's sales

Agent:

→ Generates SQL
→ Executes query
→ Returns result

3. API Tools

Connect to external applications.

Examples:

- Weather API
- Stock API
- Payment API

User:

What's the weather in Bangalore?

Agent:

→ Calls weather API
→ Returns live weather

Tool Calling Flow

Example:

User:

What is the stock price of TCS today?

Step 1: Reason

LLM thinks:

I need real-time stock data.

Step 2: Select Tool

get_stock_price("TCS")

Step 3: Execute

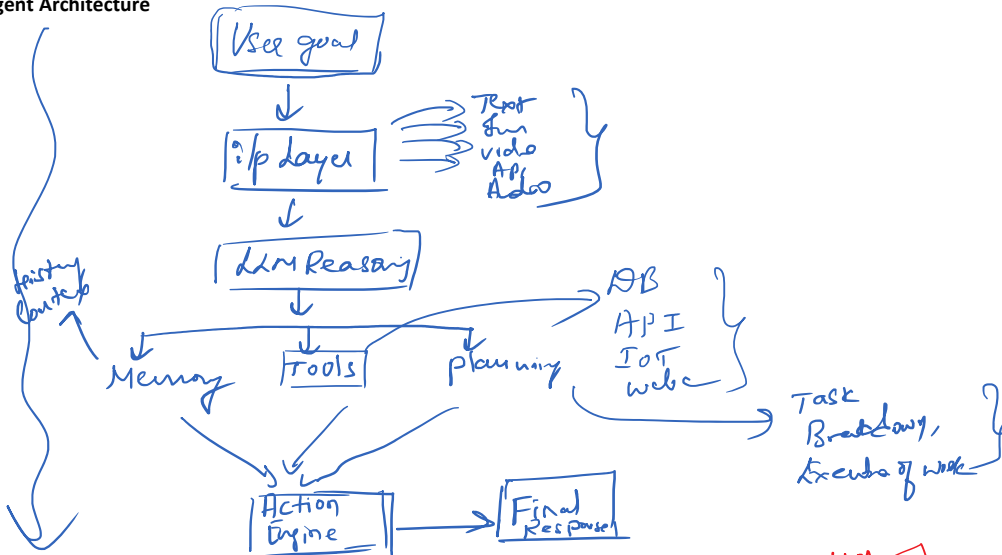
Tool returns:

```
{
  "price": 4250
}
```

Step 4: Generate Response

The current stock price of TCS is ₹4250.

AI Agent Architecture



Memory in AI Agents

Think of an AI agent like a human.

1. Short-Term Memory (Working Memory)

Information the agent remembers only during the current conversation or task.

Human Example

You ask me:

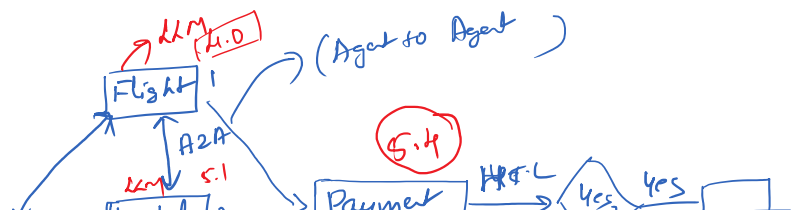
"My name is Vinay."

Later in the same conversation:

"What is my name?"

I answer:

"Vinay"



Human Example

You ask me:
"My name is Vinay."
Later in the same conversation:
"What is my name?"
I answer:
"Vinay."

I'm using short-term memory.

Agent Example

User: Find flights to Bangalore.
User: Show only evening flights.
The agent remembers the previous request and filters the flights.

Characteristics

- Temporary
- Exists only for current session
- Limited size
- Fast access

2. Long-Term Memory

Information stored permanently and reused later.

Human Example

You learned riding a bicycle 10 years ago.
You still remember it today.

Agent Example

An AI assistant remembers:
• User's favorite programming language = Python
• User works as Data Engineer
• User prefers GCP

Even after weeks, it can use this information.

Characteristics

- Persistent
- Stored in databases/vector stores
- Retrieved when needed

Types of Long-Term Memory

Long-term memory is usually divided into:

3. Episodic Memory

Memory of experiences or events.

Human Example

You remember:
• Your first day at college
• Your last interview
• A trip to Sakleshpur

Agent Example

2026-06-10
User built a RAG application using Qdrant.
2026-06-15
User deployed project on Cloud Run.
The agent remembers specific interactions and experiences.

Stores

- Conversations
- Actions taken
- Events
- Past decisions

Question

"Have I deployed any RAG project before?"

Agent checks episodic memory and answers:
"Yes, you deployed a RAG application on Cloud Run."

4. Semantic Memory

Memory of facts, knowledge, and concepts.

Human Example

You know:
• Bangalore is in Karnataka.
• Python is a programming language.
• Earth revolves around the Sun.

These are facts, not personal experiences.

Agent Example

The agent stores:
• LangChain is a framework for LLM applications.
• RAG = Retrieval Augmented Generation
• Vector databases store embeddings.

Stores

- Facts
- Concepts
- Relationships
- Knowledge

What is LangChain?

LangChain is an open-source framework used to build applications powered by large language models (LLMs) such as GPT, Gemini, Claude, and others. It helps developers connect LLMs with tools, databases, APIs, memory, and workflows.

What Langchain Provides

1. Model Abstraction

Without LangChain:

OpenAI API
Gemini API
Claude API

Different code for each model.

With LangChain:

llm = ChatOpenAI()
llm = ChatGoogleGenerativeAI()
llm = ChatAnthropic()

Same interface:

response = llm.invoke("What is AI?")

Switch models with minimal code changes.

2. Tool Framework

Tools are functions the LLM can call.

Example:

```
@tool
def get_weather(city):
    return "25°C"
```

Agent:

User

Agent

Weather Tool

Answer

Examples:

- Weather API
- SQL Database
- Search Engine
- Calculator
- Email Sender

3. Memory

Stores conversation context.

Without Memory:

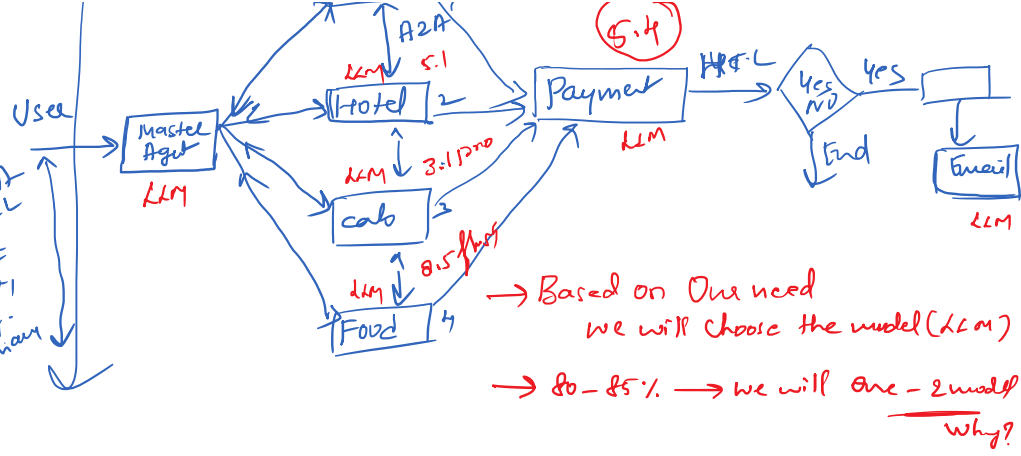
User: My name is Vinay

User: What is my name?

LLM: I don't know

With Memory:

LLM: Your name is Vinay



State Management in Agentic AI (Deep Dive)

What is State?

State = Everything the agent needs to remember while executing a task.

Think of it as the agent's working memory.

Example:

User says: "Book a flight from Bangalore to Delhi for tomorrow."

The agent needs to remember:

```
{
  "source": "Bangalore",
  "destination": "Delhi",
  "date": "Tomorrow",
  "step": "searching_flights"
}
```

This stored information is called state.

Why Do We Need State?

Without state:

User → Find flights

Agent → Found 5 flights

User → Book the cheapest one

Agent → Which flight?

The agent forgot the previous step.

With state:

User → Find flights

Agent → Found 5 flights

User → Book the cheapest one

Agent → Booking Indigo Flight 6E123

The agent remembers.

Types of State

1. Conversation State

Stores chat history.

state = {

"messages": [

"Hi",

"My name is Vinay",

...

Stores conversation context.

Without Memory:
User: My name is Vinay
User: What is my name?
LLM: I don't know

With Memory:
LLM: Your name is Vinay

- Types:
- Short-term Memory
 - Long-term Memory
 - Episodic Memory
 - Semantic Memory

4. State Management

Basic state can be maintained.

Example:

```
{
  "user_name": "Vinay",
  "cart_items": ["Pizza"],
  "step": "payment"
}
```

For complex state management:

LangGraph is designed for:

- Agent state
- Checkpoints
- Human approval
- Multi-agent systems

1. Conversation State

Stores chat history.

state = {

```
"messages": [
  "Hi",
  "My name is Vinay",
  "Recommend a laptop"
]
```

Used in chatbots.

2. Workflow State

Tracks where the process currently is.

Example:

state = {

```
"current_step": "payment"
```

Workflow:

Search Product

Add to Cart

Payment

Confirmation

If the system crashes, it can resume from "payment".

3. Tool State

Stores tool outputs.

Example:

state = {

```
"weather": "25°C",
"location": "Bangalore"
```

Other tools can reuse this information.

4. User State

Stores user preferences.

Example:

state = {

```
"favorite_food": "Biryani",
"budget": 500
```

Useful for personalization.

5. Workflow Orchestration

LangChain Chains:

Question

Retrieve Docs

Generate Answer

Validate

Return

Example:

```
chain = (
  retriever
  | prompt
  | llm
)
```

This is orchestration of multiple steps.

6. Multi-Step Reasoning

Example:

User:

Find cheapest flight and summarize hotel options.

Agent:

Step 1: Search Flights

Step 2: Compare Prices

Step 3: Search Hotels

Step 4: Rank Hotels

Step 5: Generate Recommendation

The LLM reasons through multiple steps.

Implemented using:

- Agents
- Tools
- Chains
- LangGraph

Prompt Templates in LangChain

Theory

What is a Prompt?

A prompt is the instruction we send to an LLM.

Example:

Explain Artificial Intelligence.

The model reads the prompt and generates a response.

1. Static Prompts

A static prompt never changes.

Example:

prompt = "Explain Artificial Intelligence."

Every user gets the same prompt.

Flow

Static Prompt

↓

LLM

↓

Response

Advantages

✓ Simple

✓ Easy to implement

Disadvantages

✗ Not reusable

✗ Hardcoded values

Example

```
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model="gpt-5.5")
```

```
response = llm.invoke(
```

```
"Explain Artificial Intelligence"
```

```
)
```

```
print(response.content)
```

2. Dynamic Prompts

A dynamic prompt contains variables.

Example:

Explain {topic}

The value of topic changes at runtime.

Flow

Topic

↓

Prompt Template

↓

Generated Prompt

↓

LLM

↓

Response

Example

Input:

topic = Machine Learning

Generated Prompt:

Explain Machine Learning

Why Dynamic Prompts?

Imagine teaching 100 topics.

Without dynamic prompts:

"Explain AI"

"Explain ML"

"Explain Deep Learning"

"Explain NLP"

Lots of duplicated code.

With dynamic prompts:

"Explain {topic}"

Reusable for any topic.

"Explain ML"
"Explain Deep Learning"
"Explain NLP"
Lots of duplicated code.
With dynamic prompts:
"Explain {topic}"
Reusable for any topic.



Key Takeaway

Static Prompt → Fixed text
Dynamic Prompt → Uses variables
Prompt Engineering → Improves response quality
PromptTemplate → LangChain's way to create reusable dynamic prompts.

Output Parsers in LangChain

Theory

What is an Output Parser?

An Output Parser converts the raw LLM response into a format your application can easily use.

Without parsing:

LLM
↓
Raw Text
↓
Application

With parsing:

LLM
↓
Output Parser
↓
Structured Output
↓
Application

Why Parsing is Needed?

Imagine you ask GPT:

Give me 3 programming languages.

Response:

1. Python
2. Java
3. JavaScript

This is fine for humans but difficult for applications to process.

Applications often need:

```
{  
  "languages": [  
    "Python",  
    "Java",  
    "JavaScript"  
  ]  
}
```

Output parsers help convert model output into predictable formats.

Problems Without Parsing

Example

```
response = llm.invoke(  
    "Generate 5 interview questions"  
)
```

```
print(response.content)
```

Output may vary:

1. What is AI?
2. What is ML?

or

Q1: What is AI?

Q2: What is ML?

or

- What is AI?

- What is ML?

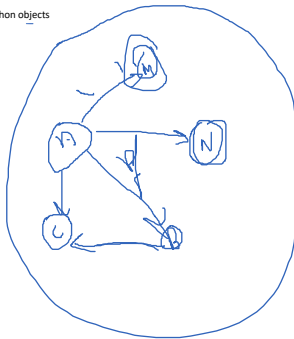
Different formats make automation difficult.

Benefits of Output Parsers

- Consistent output → JSON list tuple
- Easier automation →
- Better integration with APIs → JSON
- Reduced post-processing code →
- Structured responses →

Types of Output Parsers

Parser	Purpose
StrOutputParser	Convert output to <u>string</u>
JsonOutputParser	Convert output to JSON
PydanticOutputParser	Convert output to Python objects
Custom Parser	User-defined parsing



ReAct (Reason + Act) in Large Language Models

What is ReAct?

ReAct (Reason + Act) is a prompting and agent framework that enables a Large Language Model (LLM) to reason about a problem and perform actions using external tools to solve it.

Instead of directly generating an answer, the model follows a cycle of:

Think → Act → Observe → Think → Answer

The ReAct framework was introduced to combine the strengths of reasoning and tool usage, allowing LLMs to solve complex and real-world tasks.

Why is ReAct Needed?

Traditional LLMs have several limitations:

- They rely only on information learned during training.
- They cannot access real-time information.
- They cannot interact with external systems.
- They may hallucinate when knowledge is missing.

Example:

Question:

What is the weather in Bangalore right now?

A standard LLM cannot reliably answer this because the information changes continuously.

ReAct solves this problem by allowing the LLM to:

1. Determine what information is required.

Chains in LangChain

What is a Chain?

A Chain is a sequence of operations where the output of one step becomes the input of the next step.

Input

↓

Step 1

↓

Step 2

↓

Step 3

↓

Output

In LangChain, chains allow you to combine prompts, LLMs, parsers, retrievers, and tools into a single workflow.

Why Do We Need Chains?

Without a chain:

```
prompt_text = prompt.format(topic="Machine Learning")
```

```
response = llm.invoke(prompt_text)
```

```
answer = response.content
```

```
print(answer)
```

You manually handle every step.

With a chain:

```
chain = prompt | llm | StrOutputParser()
```

```
response = chain.invoke({
```

```
    "topic": "Machine Learning"
```

```
})
```

LangChain automatically passes data between components.

Chain Flow

User Input

↓

Prompt Template

↓

LLM

↓

Output Parser

↓

Final Output

Each component performs one specific task.

Benefits of Chains

Reusability

Create once and use many times.

```
chain.invoke({"topic": "AI"})
```

```
chain.invoke({"topic": "Machine Learning"})
```

```
chain.invoke({"topic": "Deep Learning"})
```

Readability

Instead of many lines of code:

```
prompt_text = prompt.format(...)
```

```
response = llm.invoke(...)
```

```
answer = response.content
```

Use:

```
chain = prompt | llm | StrOutputParser()
```

Maintainability

Need to change the model?

```
llm = ChatOpenAI(model="gpt-5.5")
```

No need to modify the rest of the workflow.

Output:

1. What is LangChain?

2. What is LCEL?

3. What are Chains?

- They may hallucinate when knowledge is missing.

Example:

Question:

What is the weather in Bangalore right now?

A standard LLM cannot reliably answer this because the information changes continuously.

ReAct solves this problem by allowing the LLM to:

1. Determine what information is required.
2. Call an external tool.
3. Analyze the tool's output.
4. Continue reasoning until the problem is solved.

Core Idea of ReAct

The ReAct framework alternates between:

Reasoning

The LLM thinks about the problem and decides what to do next.

Acting

The LLM executes an action using an external tool such as:

- Search engine
- Database
- API
- Calculator
- Vector database
- Knowledge base

ReAct Workflow

User Question

↓

Thought

↓

Action

↓

Tool Execution

↓

Observation

↓

Thought

↓

Action

↓

Observation

↓

Final Answer

Components of ReAct

1. Question

The user's query.

Example:

Who invented Python and what is his age?

2. Thought

The LLM reasons about the next step.

Example:

I need to find who invented Python.

3. Action

The LLM selects and executes a tool.

Example:

Search("Who invented Python?")

4. Observation

The tool returns information.

Example:

Guido van Rossum

5. Final Answer

After collecting sufficient information, the LLM generates the final response.

Example:

Python was invented by Guido van Rossum.

Example 1: Single Tool Call

Question

Who invented Python?

ReAct Process

Thought:

I need to search for the inventor of Python.

Action:

Search("Who invented Python")

Observation:

Guido van Rossum

Thought:

I have the required information.

Final Answer:

Python was created by Guido van Rossum.

Example 2: Multiple Reasoning Steps

Question

Who is the Prime Minister of India and what is his age?

ReAct Process

Thought:

First, I need to identify the Prime Minister of India.

Action:

Search("Current Prime Minister of India")

Observation:

Narendra Modi

Thought:

Now I need to determine Narendra Modi's age.

Action:

Search("Narendra Modi age")

Observation:

75 years

Thought:

I have enough information.

Final Answer:

The Prime Minister of India is Narendra Modi, and he is 75 years old.

ReAct Prompt Format

A standard ReAct prompt follows this structure:

Question: <user question>

Thought: <reasoning>

Action: <tool name>

Action Input: <tool input>

Observation: <tool output>

Thought: <additional reasoning>

Final Answer: <answer>

Example ReAct Prompt

Question: What is the capital of Japan?

Thought: I need to search for Japan's capital.

Action: Search

Action Input: Japan capital

Observation: Tokyo

Thought: I found the answer.

Final Answer: The capital of Japan is Tokyo.

Chain of Thought ReAct

Only reasoning Reasoning + acting

No external tools Uses external tools

Static knowledge Dynamic knowledge

Single execution Iterative execution

Chain of Thought

Step 1: How do we meaning?

User: Hi
AI: Hello,
User: My name is Vin
AI: Hi ---

Return

Step 3

System Prompt

Chat history

User: What is 20+20

Scratchpad:

(Empty)

Step 4:

prompt feed to LLM (GPT)

(Tool description) → doc string.

Steps:

(LLM)

GPT decide tool

"Evaluate Math Exp"

I should use calculator tool

tool call

{ name: calculator

args:

expres

20+20

}

} LLM Return

Step 6: Agent Executes Return it

Step 7: @tool

def calculate ()

Return(40)

Step 8:

Scratch pad

!p 20+20

!p 40.

Step 9: langchain built the new prompt.

System Prompt

Chat history

=====

User Q

20+20

tool

calculator

!p 20+20

!p 40

Step 10:

!p → 40

↓

!p T

40

Step 11:

User what is 20+20

AI: The ans is 40

} → Update
↓
add to existing chat

Chain of Thought ReAct	
Only reasoning	Reasoning + acting
No external tools	Uses external tools
Static knowledge	Dynamic knowledge
Single execution	Iterative execution

Chain of Thought

Question
↓
Reason
↓
Answer

ReAct

Question
↓
Reason
↓
Action
↓
Observation
↓
Reason
↓
Answer

Applications of ReAct

- ReAct is widely used in:
- Retrieval-Augmented Generation (RAG)
 - Search agents
 - Web browsing agents
 - Database query agents
 - Customer support systems
 - Travel assistants
 - Financial assistants
 - Code generation agents
 - Autonomous AI agents

Advantages of ReAct

- Reduces hallucinations
- Supports real-time information retrieval
- Enables multi-step reasoning
- Allows interaction with external systems
- Produces more reliable answers
- Improves transparency through reasoning traces

Limitations of ReAct

- Requires additional tool calls
- Increases latency
- Higher computational cost
- Depends on tool quality
- Complex reasoning chains may fail

